



Cairo University Racing Team

Formula Student / FSAE®

Software Development Team

(Backend - Technical Task)

Season 26-27

Team Project Management System

System Description

The system is designed to help teams organize their projects, manage tasks, and track the progress of their work. The platform allows users to create and manage projects, break projects down into tasks, assign tasks to team members, and track their status and priority.

Core Entities

Users

- Create an account and log in.
- Manage their profile.
- Participate in projects and tasks.

Projects

- Create a project with a name and description.
- View project details and progress.
- Update and delete projects.
- Associate projects with their members/tasks.

Tasks

- Create tasks within a project.
- Assign tasks to team members.
- Set a task's title, description, priority, and status.
- Update and delete tasks.
- Track task progress.

Task Status & Priority

- Status: To Do -> In progress -> Done
- Priority: Low -> Medium -> High

Tasks

Complete up to the level you're comfortable with, do as much as you can:

Level 1: Core API & Functionality

- **Set up a server** (Node/Express, or any framework you're comfortable with) and a **database** (SQL or NoSQL, your choice).
- **Define data models/schema** for Users, Projects, and Tasks.
- CRUD endpoints for **Projects** (create, read, update, delete).
- CRUD endpoints for **Tasks** (create, read, update, delete), including status and priority fields.
- **Basic input validation** on all endpoints (required fields, correct types, meaningful error messages).
- Proper **HTTP status codes** and **consistent** JSON response structure.
- **Clean project structure** (routes/controllers/models separated, not everything in one file).
 - Data can be tested via Postman/Insomnia, no auth required yet.

Level 2: Real Authentication & Data Integrity

Extend Level 1 with:

- **Real authentication:**
 - **POST /register** : create a user with a hashed password (bcrypt or similar). Never store plain-text passwords.
 - **POST /login** : verify credentials, return a JWT (or session token) on success.
 - Protect relevant routes with authentication middleware, reject requests without a valid token (401 Unauthorized).
- **Ownership enforcement:** a user can only access (view/update/delete) Projects/Tasks they own or are a member of, reject requests from unrelated users with 403.
- **Task assignment:** assign tasks to a valid member of the project (assignedTo, storing a userId that must exist and belong to that project).
- Proper error handling middleware (no raw stack traces returned to the client)

- Environment variables via a .env file (DB connection string, JWT secret, port, ...), include a file like .env.sample or show how to configure these vars in README file.
- Pagination and basic filtering on list endpoints (e.g. GET /tasks?status=done)

Level 3: Advanced Back-end Application

Extend Level 2 with:

- **Role-based access control (two roles only):**
 - **Owner** : the user who created the project. Can edit/delete the project, add/remove members, and edit/delete any task in it.
 - **Member** : a user added to the project. Can view the project and update the status of tasks assigned to them, but cannot edit the project or delete tasks.
 - Enforce this on the backend with middleware : attempts by a Member to perform Owner-only actions must return 403 Forbidden.
- Advanced filtering/searching/sorting on Projects and Tasks (by status, priority, assignee, date, etc.)
- Rate limiting on sensitive endpoints (e.g. login) to prevent brute-force attempts
- Input sanitization to prevent injection attacks
- Logging (requests, errors) using a proper logger (e.g. Winston/Morgan), not just console.log
- Unit and/or integration tests for at least the core endpoints (auth, CRUD)
- API documentation (Swagger/OpenAPI, or a well-structured Postman collection)

General Instructions

Submission

- Submission is via a GitHub public repository.
- A README.md is required and should include: project overview, technologies used, setup instructions, how to run the project, database setup/migration steps, API documentation/endpoints list, database design, and any assumptions or additional features implemented.

- Deploy your project (e.g. Vercel, Render or Railway, or similar) and include the live API base URL in the README. If for any reason it's not deployed, the README must clearly explain how to run it locally instead.
- A short explanation video (5-10 mins) is required, walk through your app and briefly explain your code structure, it must be submitted via Google Drive **(Make sure it's public)**
- *Send an email with the subject "Software team Task Inquiry", including : Github repo link, Deployed Link, .env file and your name.*

General Rules

- Searching online and using external resources (docs, Stack Overflow, tutorials) is allowed.
- You can use any framework you are comfortable with.
- Using AI tools to generate code is not allowed. The implementation must be your own understanding and your own work, you may be asked to explain or modify any part of your code in person, and AI-generated code you can't explain will be treated as a violation.
- The project should be completed within 5 days.
- Commit history will be reviewed: commit incrementally, not as one giant commit.
- Seed the app with mock/sample data (users, projects, tasks) so it can be viewed and tested immediately in the demo video or by reviewers, don't submit with empty data.
- Be prepared for a follow-up Q&A / code walkthrough after submission. You may be asked to explain any part of your implementation, justify design decisions, or make a small live change to your code.
- For any questions during the task, contact us via **sda.curt@gmail.com**